Document number: P1018r4
Audience: WG21

Ville Voutilainen
2019-08-04

Evolution status after Cologne 2019

Abstract

This paper is a collection of items Evolution has worked on up to and in the Cologne meeting, their status, and plans for the future.

Contracts

Evolution discussed multiple different approaches for Contracts for C++20; after initially choosing to pursue P1607R0 Minimizing Contracts, decided to go with D1823R0 Remove Contracts from C++20. Expecting Contracts to make a comeback for C++23.

Coroutines

Evolution discussed P1745R0 Coroutine changes for C++20 and beyond, and P1485R1 Better keywords for the Coroutines; there was no consensus to adopt either of these.

Modules

Evolution approved what can be considered as bugfixes and a minimal amount of library support for Modules; these are

- D1811R0 Relaxing redefinition restrictions for re-exportation robustness,
- P1502R0 Standard library header units for C++20,
- P1703R0 Recognizing Header Unit Imports Requires Full Preprocessing,
- and P1766R0 Mitigating minor modules maladies.

All Your Spaceships Are Belong to Us

Evolution approved P1630R0 Spaceship needs a tune-up: Addressing some discovered issues with P0515 and P1185. This makes the reversed operator candidates be required to return bool, and deletes defaulted operators for types with reference members or variant members.

Concepts

Evolution approved P1452R1 On the non-uniform semantics of return-type-requirements; this removes an expression constraint -> Type, and C++ now necessitates the use of -> std::same or -> std::convertible_to. Evolution also approved P1616R0 Using unconstrained template template parameters with constrained templates.

A couple of small fixes

Evolution approved

- P1668R0 Enabling constexpr Intrinsics By Permitting Unevaluated inline-assembly in constexpr Functions,
- P1675R0 rethrow_exception must be allowed to copy,
- and P1771R0 nodiscard for constructors.

Floating-point types as non-type template parameters

Evolution approved P1714R0 NTTP are incomplete without float, double, and long double!. This allows floating point types as NTTPs, by making their comparison be done in terms of bit representation. While this is somewhat novel and late, it's allegedly possible to write a work-around in C++20 with a fairly small amount of code, and Evolution elected to support floating-point types directly.

Near-future EWG plans

The high-order bit of our next steps is ballot resolution in Belfast. Once that is done, Evolution will switch into full C++23 mode.